

Article

Cybersecurity in Smart Grids: Detecting False Data Injection Attacks Utilizing Supervised Machine Learning Techniques

Anwer Shees, Mohd Tariq and Arif I. Sarwat *

Department of Electrical and Computer Engineering, Florida International University, Miami, FL 33174, USA; ashee021@fiu.edu (A.S.); tmohd@fiu.edu (M.T.)

* Correspondence: asarwat@fiu.edu

Abstract: By integrating advanced technologies and data-driven systems in smart grids, there has been a significant revolution in the energy distribution sector, bringing a new era of efficiency and sustainability. Nevertheless, with this advancement comes vulnerability, particularly in the form of cyber threats, which have the potential to damage critical infrastructure. False data injection attacks are among the threats to the cyber–physical layer of smart grids. False data injection attacks pose a significant risk, manipulating the data in the control system layer to compromise the grid’s integrity. An early detection and mitigation of such cyberattacks are crucial to ensuring the smart grid operates securely and reliably. In this research paper, we demonstrate different machine learning classification models for detecting false data injection attacks, including the Extra Tree, Random Forest, Extreme Gradient Boosting, Logistic Regression, Decision Tree, and Bagging Classifiers, to secure the integrity of smart grids. A comprehensive dataset of various attack scenarios provides insights to explore and develop effective detection models. Results show that the Extra Tree, Random Forest, and Extreme Gradient Boosting models’ accuracy in detecting the attack outperformed the existing literature, an achieving accuracy of 98%, 97%, and 97%, respectively.

Keywords: smart grid; false data injection attack; cyber attack; machine learning



Citation: Shees, A.; Tariq, M.; Sarwat, A.I. Cybersecurity in Smart Grids: Detecting False Data Injection Attacks Utilizing Supervised Machine Learning Techniques. *Energies* **2024**, *17*, 5870. <https://doi.org/10.3390/en17235870>

Received: 3 October 2024

Revised: 14 November 2024

Accepted: 17 November 2024

Published: 22 November 2024



Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

In the past decade, the transformation of grid distribution networks into smart grids has shown a major advancement in technology, stretching forth the smooth convergence of sustainability, dependability, and energy efficiency for years in the future. To control, monitor, optimize, and predict the production of energy through smart grid infrastructure, there is a requirement for cutting-edge technology [1,2]. The complete system could easily be turned into a new era of linked, data-driven power grids for the use and management of electricity. However, with all of these benefits, we rely on digital infrastructure and a lot of data exchanges, which brings a wider spectrum of cyberthreat vulnerabilities. Weak points in the networked system and devices are prone to such dangers, which make up the whole infrastructure of smart grids [3].

False data injection attacks (FDIAs) are a different type of cyberthreat, tailored specifically to disrupt the smooth functioning of smart grids [4]. These attacks involve false or altered data into the grid’s control system to make the measurements imprecise, make erroneous decision calls, and perhaps disastrously cause the operation to fail [5]. This attack can interfere with the regular functioning of smart grids, harm data accuracy, and perhaps cause widespread power outages by weakening the power and integrity of data within the system. The consequences of these attacks are profound, affecting not only the utilities but also millions of consumers who pay for uninterrupted electric power for their chores. And, therefore, it is highly essential to counter false data injection attacks before the evil attackers grow craftier and more determined [6]. The adverse impact on a nation’s total well-being and survival, including its economy, health, and security, will be detrimental

if the smart grid infrastructure is successfully attacked. The false data injection attack is one of those that are most dangerous to smart grids. In 2015, using spear-phishing emails, the Black Energy virus gained access to the control systems of multiple regional electricity distribution companies. The virus injected false data into the systems, manipulating sensor readings to obscure the actual status of the power grid, affecting Ukraine's power grid [7]. In 2016, a specially designed piece of malware called Industroyer attacked a Ukrainian transmission facility, compromising the Industrial Control System and causing an hour-long power outage. In June 2021, phishing and remote vulnerabilities were used as attack vectors against Florida Municipal Power agencies in the United States. Although the attackers were able to obtain some degree of access, the attempt was stopped before it could have disastrous consequences [8]. Securing and maintaining smooth functioning depend on our capacity to detect and mitigate such attacks as early as possible [9,10]. Detecting attack patterns and unusual behavior in the complex, dynamic, and massive amount of data a smart grid produces is an arduous challenge. For better visualization, in Figure 1, we portray a smart grid set up under a false data injection attack (FDIA) scenario.

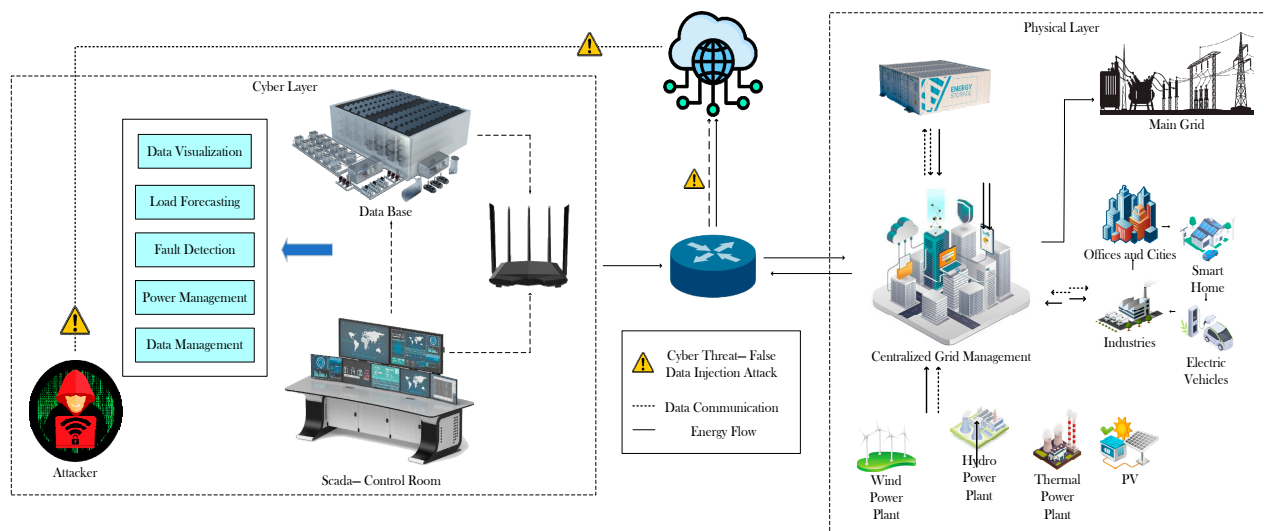


Figure 1. Smart grid under FDIA scenario in the Cyber Layer.

To address this pressing challenge of terminating false data injection attacks (FDIAs) [11–14], this research explores different classification modeling approaches with the goal of arming smart grids with a robust system for the early detection of attacks and protect them by using different machine learning algorithms. The main aim is to use the data-driven approach and machine learning algorithms to detect any attacks and secure the grid before attackers have any opportunity to compromise the grid. A vast dataset of false data injection attack scenarios in smart grids is at the root of this research [15]. The magic behind training the models that can accurately classify the attack event and normal events is in the datasets that we have used; the datasets have been carefully selected from actual attack events and artificial attack event patterns. With the proposed approaches and insight gleaned from the datasets, the research paper contributes to enhancing the cybersecurity of smart grids.

In the pages that follow, we will embark on a comprehensive journey through the landscape of smart grid cybersecurity in other literature in Section 2. The methodology used in this research shall be discussed in Section 3, which includes the workflow of this work, as well as different machine learning techniques and details of their hyper-tuning parameters. Following Section 3, the results are discussed in Section 4 for different techniques; lastly, in Section 5, we have provided a detailed discussion on the dataset utilized in this work and made a comparison with other works that have similar datasets to the one here utilized.

2. Related Works

In recent years, many comprehensive machine learning studies providing highly insightful information on the use of attack modeling in the context of smart grids have been published as part of employing machine learning approaches to model data in terms of holding attacks and normal data patterns. Yohanandhan et al. [16], for instance, investigated various modeling approaches for cyber–physical power systems in addition to simulation methodologies for a variety of cyberthreats and cybersecurity safeguards. Although not fully described, machine learning for cybercrime investigations is addressed as a perspective. The cybersecurity review of smart grids published by Nejabatkhah et al.'s [17] study is more broadly focused on smart grid description, integrity attacks—specifically, false data injection attacks (FDIAs)—general security mechanisms, and the financial effects of cyberattacks on both the cyber and physical levels. In their review, the mention of machine learning is almost completely missing or absent. Ye et al. offer a discussion on cybersecurity issues and cybersecurity's potential for power grids, particularly photovoltaic systems [18]. Both the model-based and the data-driven approaches to modeling security systems have been proposed. Additionally, extended blockchain techniques are shown. Future perspectives on machine learning and artificial intelligence are briefly covered. Hossain et al.'s [19] excellent contribution in this regard is their analysis of machine learning security using both large data and machine learning. As a result, machine learning has only been considered as a single technique without being broken down into specific approaches for categorization (such as traditional and cutting-edge deep learning). In actuality, the machine learning portion is more focused on threat kinds and learning paradigms like supervised learning and unsupervised learning. Additionally, its uses in the cybersecurity of solar energy and wind energy systems are highlighted. Alimi et al.'s [20] study of machine learning explores power system's stability and cybersecurity. Detecting cyberattacks, power quality disruptions, and dynamic security evaluation are examined in relation to machine learning. Therefore, machine learning tools are included with a focus on reinforcement learning and deep learning. A thorough investigation of false data injection attack (FDIA) detection techniques in smart grid networks is provided by Musleh et al. [21]. In a brief discussion, machine learning is described as a method for detecting false data injection attacks (FDIAs) that may be implemented in three different ways: supervised learning, unsupervised learning, and reinforcement learning. Machine learning is introduced in two sub-categories, standard machine learning and advanced deep learning approaches, in a review study by Kotsiopoulos et al. [22]. These categories were examined considering how they might be used in Industry 4.0, where smart grids are more specific (for instance, embedded artificial intelligence devices, resilient factories, smart humans (SHs) and health performances, predictive energy systems, and worry-free transportation). They identified the machine learning subject as a difficulty for these applications. As a result, a fascinating and important aspect of federated learning is highlighted and has been further debated throughout time. The key area of study in Cui et al.'s work [23] is FDIA-based machine learning in smart grids. The three primary themes of load forecasting, state estimation, and fraud detection are used to guide the investigation criteria for machine learning modeling. Following that, machine learning is divided into three major groups, i.e., supervised, unsupervised, and reinforcement learning (RL). A survey of intrusion detection systems in smart grids was introduced by Jow et al. [24], after which it was indicated that machine learning adheres to many learning paradigms, including supervised learning and unsupervised learning. Another complete study-based intrusion detection and prevention system was introduced by Radoglou-Grammatikis et al. [25]. Machine learning is typically referred to as an anomaly-based approach among the three categories of intrusion detection techniques: signature-based, anomaly-based, and specification-based. In performing a brief literature review, we summarize and compare in Table 1 the amount of machine learning used and different techniques used, for instance, supervised learning, unsupervised learning, reinforcement learning, anomaly detection, etc., in the context of smart grid cybersecurity in the state of the art. This helps to understand the research gap in

the amount of machine learning techniques that are utilized for the cybersecurity of smart grids and thereby gives motivation to further move forward with this work.

Table 1. Overview of state of the art and their machine learning approaches. In the table ✔ implies machine learning techniques utilized and ✘ implies machine learning not utilized.

State-of-the-Arts	Machine Learning Involved in Context of Smart Grid's Cybersecurity	
Yohanandhan et al. [16]	At perspective level	✔
Nejabatkhah et al. [17]	No discussion	✘
Ye et al. [18]	At perspective level	✘
Hossain et al. [19]	Utilized for big data analysis	✔
Alimi et al. [20]	Utilized but not for cyberthreat	✘
Musleh et al. [21]	Supervised, Unsupervised and Reinforcement learning	✔
Kotsiopoulos et al. [22]	Challenge for machine learning in application of smart grid cybersecurity	✔
Cui et al. [23]	Supervised, Unsupervised, and Reinforcement Learning	✔
Jow et al. [24]	Supervised, Unsupervised, and Reinforcement Learning	✔
Radoglou et al. [25]	Anomaly-based Machine Learning technique is utilized	✔

The study highlights the necessity of utilizing machine learning methods to strengthen smart grid security against cyberattacks. Numerous methods are employed in the detection and mitigation of various assaults, including denial of service (DoS) and false data injection (FDI), semi-supervised anomaly detection, deep representation learning, and support vector machine (SVM) classification. These solutions ensure the integrity and stability of smart grid operations by utilizing real-time data from data servers, power flow monitoring, and smart meters to identify anomalous activity. Moreover, it is commonly accepted that effective machine learning techniques are required to effectively counteract the increasing number of cyberattacks. It is crucial to investigate potential hostile attacks and suitable defenses. Moreover, problems with efficiency and data availability can be solved by innovative detection techniques like collaborative and decentralized learning. In Figure 2, we have demonstrated the workflow of our paper.

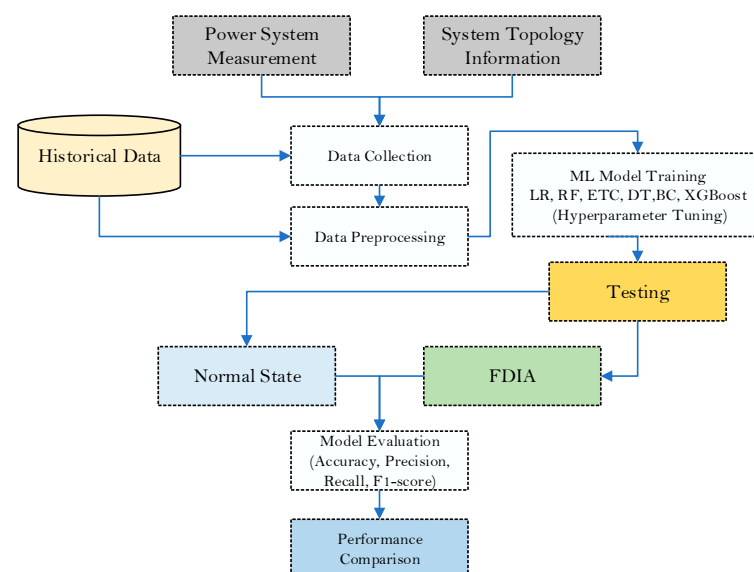


Figure 2. Flow diagram of the work conducted.

3. Machine Learning Approach

Machine learning aims to learn from data and make decisions from learning [26]. Numerous research works have been conducted on the subject of teaching computers to learn on their own without explicit programming. Numerous mathematicians and programmers use a variety of techniques to solve problems with large amounts of data. Several methods are used in machine learning to address data-related issues [27]. The optimal kind of algorithm to tackle an issue is not a one-size-fits-all solution, as data scientists prefer to emphasize. The type of method used relies on the type of issue you want to solve, how many variables there are, what form of model works best for it, and other factors. Here is a brief overview of some of the machine learning methods that are employed in this work for our purpose of FDIA detection.

3.1. Training Model

In this section, we discuss the details of the different machine learning algorithms we have implemented to detect attack events, normal events, and no events.

3.1.1. Extra Tree Classifier

The Extra Tree method combines the Decision Tree, Random Forest, and Bootstrap aggregation, also known as the bagging method [28]. It is one of the ensemble learning algorithms. The Extra Tree Classifier works by producing unpruned Decision Tree, unlike the Decision Tree Classifier or Random Forest Classifier. The interesting point is that it uses the Random Forest style for random feature sampling at each split point inside a Decision Tree for the sake of increasing the diversity of the model and lowering the overfitting like hood [29,30]. The Extra Tree algorithm takes the approach of the random selection technique for division points in contrast to the avaricious approach of the Random Forest. This feature is the core to making the Extra Tree Classifier algorithm different from the other tree-based models and highly contributes to its resilience and flexibility. The detailed steps to make a decision by the Extra Tree Classifier are shown in Figure 3.

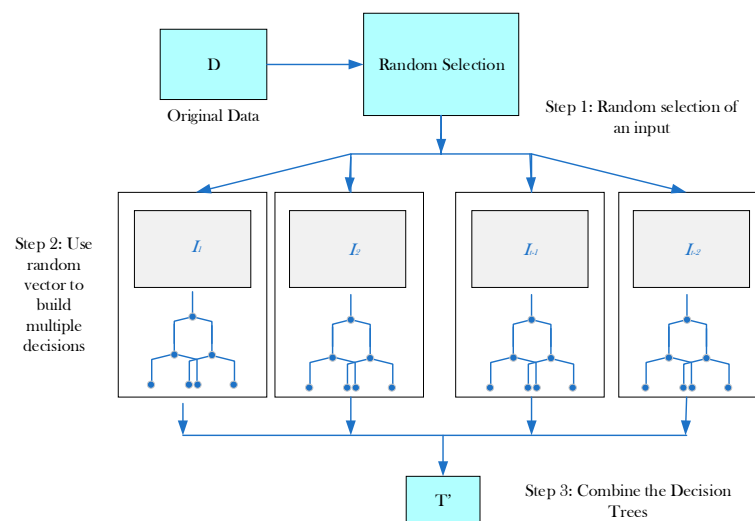


Figure 3. Process of decision-making by Extra Tree Classifier.

We have implemented the Extra Tree method through the scikit-learn package, and, by setting up hyperparameters, we shaped its functionality for our application. The N estimator indicated the number of trees that build up the forest; in our case, it is set to 100. For $n_estimator = 100$, note that, by using a higher number of estimators, we can increase the robustness of the model [31]. However, it will trade the complexity of the model as well. To evaluate the split quantity of the model, we use Gini metrics, indicated as $creation = Gini$. Gini is measuring the impurity that influences the tree's branching to make the best classification accuracy through identifying large discrepancies in the label probability

distribution inside the nodes, which forms the child nodes in the algorithm. To further divide the internal nodes, the minimum number of samples required is two parameters, indicated as `min_sample_split` = two parameters. By changing this parameter, it can affect the models' breadth and depth, which further can affect the overall performance of the model and its sensitivity to individual data points. Gini is utilized in this work to assess the node impurity to its maximum, with the aim of reducing the misclassification by the Extra Tree model. This step is crucial for resulting in a better classification of the attack scenario. For the rest of the arguments, we have used the default values. It is discovered that the best result in our dataset is through optimizing parameters using the Bayesian optimizing technique [32]. We optimized `n_estimator` for the range of 50–200, and `min_sample_split` for the range of 2–10, and set the best hyperparameters. The technique makes use of Bayesian theorems for optimization; this strategy gives good performance when there is a huge and complex parameter space, and the objective function is costly to evaluate. The tuning of model hyperparameters is a challenging task as the classification problem holds intricate interactions [33]. Therefore, Bayesian optimization has proven to be a good selection as it minimizes the number of evaluations required to determine the ideal collection of hyperparameters while balancing space exploration and exploitation skillfully. This works through creating a stochastic framework for the target function; further, to choose the objective function, it is effectively analyzed through the acquisition function before selecting the potential samples. This approach lines up the rational inquiry into the global optimization problem. Bayesian optimization is utilized to optimize the hyperparameters of the model. By this technique, we reduce the complexity involved in determining the hyperparameter of the model. Bayesian optimization is chosen over other techniques solely for the purpose of its capability of efficiently balancing the search for optimal hyperparameters through combining exploration with the refinement of a promising area exploitation. It strategically identifies promising areas in the parameter space, which reduces the unnecessary evaluations, minimizes the trial and error, prevents overfitting, and improves model generalization on unseen data. This makes the best choice for our classification model. We validated Bayesian optimization for over 20% of the original dataset.

3.1.2. Random Forest Classification

The Random Forest Classifier consists of a combination of tree classifiers, with each tree classifier generating using a random vector sampled individually from the input vector, which in combination makes up the Random Forest Classifier. Each tree calculates a unit vote for the most popular class to classify an input vector [34]. The Gini Index, an attribute selection measure that measures the impurity of an attribute according to the classes, is adapted by the Random Forest Classifier. For a given training set T , select one case (pixel) at random and say it applies to class C_i . The Gini Index can be written as Formula (1):

$$\sum \sum_{j \neq i} \left(\frac{f(C_i T)}{|T|} \right) \left(\frac{f(C_j T)}{|T|} \right) \quad (1)$$

where $\left(\frac{f(C_i T)}{|T|} \right)$ is the probability that the selected case applies to class C_i .

A Random Forest (RF) tree is made up of several classifiers, each of which contributes one vote to determine which class is assigned to the input vector (x) most frequently. The formula for this is $C_{rf}^B = \text{majority vote } \{C_B(x)\}_1^B$ where $C_B(x)$ is the class prediction of the B th Random Forest tree. Since the Random Forest (RF) model possesses unique qualities not seen in a typical classification tree (CT) due to its amalgamation of several classifiers, it needs to be viewed as a novel idea in the field of classifiers. By forcing the trees to grow from various training data subsets produced by bagging or bootstrap aggregating, a Random Forest (RF) increases the variety of the trees. By randomly resampling the original dataset with replacement (i.e., without deleting the data taken from the input sample for producing the next subset), a technique known as bootstrap aggregating is employed for training data production. Using trees as base classifiers, $\{h(x, \Theta_k), k = 1, \dots\}$, where x is

the input vector and $\{\Theta_k\}$ are the independent and identically distributed random vectors, is how the Random Forest (RF) ensemble classification technique operates [35]. As a result, some data might be used more than once to train classifiers, while other data might not be used at all. More classifier stability is thus attained, which simultaneously improves classification accuracy and strengthens the classifier's resistance to small changes in input data [35]. Numerous studies have shown that, in contrast to other boosting strategies, bagging methods (like RF) are not susceptible to noise or overtraining [35–40].

3.1.3. Extreme Gradient Boosting Classifier

The Extreme Gradient Boosting (XGBoost) Classifier algorithm, which is proposed by Pedregosa et al. [41] in his work, is in general used for classification. This algorithm is an open-source library or framework, unlike other algorithms. It is probably an algorithm-boosting framework. The details are given in Equation (2).

$$y_i^{(t)} = \sum_{k=1}^t f_k(x_i) = y_i^{(t+1)} + f_k(x_i) \quad (2)$$

The regularization strategies of the algorithm prove to make the algorithm more flexible in a range of learning contexts, faster than gradient boosting, and more compatible. Also, parallel computing makes the outcomes run faster in the time-sensitive situation [42]. In this work, the Xgboost package is used and implemented through Python version 3 to develop the classifier. The model is designed according to the category's expected likelihood of the input data. A further gbtrees booster is used as the classifier's booster core and goes through multiple experiments for the best classification results, with 0.3 being the learning rate for the best case. To determine the optimal way to represent the features using automated machine learning (AutoML), we first scale the data and then do the encoding using different techniques. Through automated machine learning, we may effortlessly find the high-performing machine learning model pipeline for the classification modeling assignment. This study utilizes two primary Hyperopt-Sklearn and TPOT libraries of AutoML. To model the configuration of the model, the Hyperopt-Sklearn uses Bayesian optimization, and genetic programming is used by the TPOT for searching through the vast space of potential pipelines. These techniques are employed in this study to increase the classification of the attack scenario to determine the optimal feature representation technique. The results showcase that the encoding techniques did not contribute significantly to improving the quality of classification; therefore, in the study, they were not implemented as the initial implementation for the sake of lowering the complexity of the model time. However, in the performance of the model trained on the dataset, there is no discernible difference between employing encoding techniques. Moreover, unlike the influence of machine learning algorithms and model hyperparameters in the quality of classification, encoding techniques did not perform well.

3.1.4. Logistic Regression

In the area of the cyber security of smart grids, it is crucial to classify between normal events and attack events to ensure the integrity and security of the system. Among them, false data injection is a potential threat to the system, and hence classification is paramount. Considering this, Logistic Regression is a powerful tool for such classification tasks due to its simplicity, effectiveness, and interoperability. It is widely used in the binary classification problem due to its statistical methods, where there are typically only two output categories, 0 (normal) and 1 (attack). The false data injection attack (FDIA) application, therefore, seamlessly aligns with the functioning of this algorithm. To understand better the algorithm, Equation (3) represents the input output function. The heart of Logistic Regression lies on the sigmoid function denoted as $\sigma(z)$, where z is a linear combination of input features:

$$\sigma(z) = \frac{1}{1 + e^{-x}} \quad (3)$$

In the above equation, e represents the base of the natural logarithm. The logistic function shown maps any real valued number to the range of $[0, 1]$.

For more inputs, the Logistic Regression models a linear combination, and, to obtain the probabilities, it is transformed into a logistic function. The model is represented by Equation (4), given as follows:

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n \quad (4)$$

where

z represents the linear combination of input features and coefficients.

x_0, x_1 and x_2 are the input features.

$\beta_0, \beta_1, \beta_2, \dots, \beta_n$ are the coefficients (parameters) to be learned

To estimate the probability, further the logistic function is applied to the linear combination, through which we obtain the probability of the positive class (attack scenario in this case), as shown in Equation (5):

$$P(y = 1 | x) = \sigma(z) = \frac{1}{1 + e^{-x}} \quad (5)$$

Similarly, for the negative class (normal scenario), the probability can be expressed as Equation (6):

$$P(y = 0 | x) = 1 - P(y = 1 | x) \quad (6)$$

For the model training, through maximum likelihood estimation or gradient descent, the coefficients $\beta_0, \beta_1, \beta_2, \dots, \beta_n$ are estimated for optimization techniques for the purpose of minimizing the error between predicted probabilities and actual class labels.

In the Logistic Regression algorithm, the feature space is separated through the decision boundaries. Decision boundaries are hyperplanes that separate different classes and are determined using the coefficient learned during the training. For the study, Logistic Regression offers a great solution through its robust framework that leverages the logistic function to model the probabilities and linear combination of features to classify the false data injection event. Its simplicity, low computational time, and solid mathematical foundation make the choice attractive for addressing the challenges posed by a false data injection attack (FDIA).

3.1.5. Decision Tree

The Decision Tree model provides a transparent and interpretable framework for our classification task of attacks and normal events; it is essential and pivotal to detect false data injection attacks (FDIAs) to maintain the integrity of smart grids. The Decision Tree model relies on solid mathematical principles to partition the feature space effectively and is utilized in general for classification and regression tasks. It is a non-parametric, supervised learning model. The model's working principle is that it recursively partitions the feature space into disjoint regions, each of which corresponds to a particular label. The Decision Tree algorithm makes decisions by maximizing the information gain or minimizing the impurity, which is shown mathematically in the equations below.

The entropy $H(S)$ measures the amount of disorder or uncertainty in a particular set of data points, while the information gain quantifies the reduction in entropy achieved by splitting the data based on a feature. $H(S)$ for the set S with class proportions p_i is calculated in Equation (7).

$$H(S) = - \sum_i p_i \log_2(p_i) \quad (7)$$

In Equation (7), p_i represents the probability of each possible outcome occurring.

The information gain $IG(X, Y)$ is calculated in Equation (8). It is the difference between the entropy of the parent node and the weighted sum of entropies of the child nodes, used for splitting in feature X with class labels Y .

$$IG(X, Y) = H(Y) - \sum_i \frac{|S_i|}{|S|} H(S_i) \quad (8)$$

In Equation (8), S_i represents the subset of data points at the i_{th} child node, and $|S|$ is the total number of data points.

Through Gini impurity, we measure the probability of misclassifying a randomly chosen element in the dataset. It is calculated as shown in Equation (9). The Gini impurity for a split is the weighted sum of the impurities of the child nodes. For a set S with class proportions p_i , Gini impurity $I_G(S)$ is calculated as follows:

$$I_G(S) = 1 - \sum_i p_i^2 \quad (9)$$

In Equation (9), the p_i represents the probability of each possible outcome occurring.

To determine the best feature and threshold for data partitioning, the Decision Tree employs a splitting criterion based upon the entropy and information gain or Gini impurity at each internal node. Then, the decision rule is applied to each internal node to follow the branch based on the value of specific features. Mathematically, the decision rule can be represented as shown in Equation (10).

$$X_j \leq t \quad (10)$$

where X_j is the value of feature j , and t is the threshold for splitting.

We have controlled the model complexity by limiting the maximum depth of the tree to prevent overfitting. Overall, the mathematical principle of the Decision Tree, i.e., entropy, information gain, and Gini impurity, leverages to build a hierarchical decision-making structure for the application of false data injection classification.

3.1.6. Bagging Classifier

In order to safeguard the smart grid from malicious activities such as false data injection attacks, we require a robust classification model for such tasks as the integrity of the grid is paramount, which can be dealt with by another ensemble learning method, the Bagging (Bootstrap Aggregating) Classifier. As mentioned earlier, it is an ensemble classifier that aims to improve the stability and accuracy of the model through combining the classification of multiple base learners. The functioning is based on generating multiple bootstrap samples from the original dataset, training a base learner on each sample, and then aggregating their predictions to make the final decision for classification. The mathematical foundation of a Bagging Classifier starts with bootstrap sampling, creating multiple bootstrap samples through random sample observation and replacing them with the original dataset.

To represent this mathematically, let D denote the original dataset with N observations. A bootstrap sample D_i of size N_i is obtained by sampling N_i observations from D with replacement, and $h_i(x)$ represents the prediction made by the i th base learner for input x . The base learner is trained on each bootstrap sample D_i .

The classification is aggregated for making final decisions after all of the base learners are trained. A usual aggregation technique, the weighted majority voting scheme, is used in the context of classifying the false data injection attack, where the label with the highest majority voting across the base learners is selected to make the final decision. Each base learner is assigned a weight based upon its performance level. Let w_i denote the weight assigned to the i th base learner's prediction. The final prediction is then determined as shown in Equation (11):

$$\hat{y} = \operatorname{argmax}_y \sum_{i=1}^M w_i \cdot I(h_i(x) = y) \quad (11)$$

where M is the total number of base learners, $I(\cdot)$ is the indicator function, and $\hat{y}$ is the final predicted class label.

We generalize the model through training multiple bases on the different bootstrap samples, thus reducing the overfitting and enhancing the generalization ability of the ensemble learning model. This promotes the diversity of models among base learners, which leads to a more robust classifier capable of handling unseen data. The Bagging Classifier proves to be a power framework for the false data injection attack task, leveraging the ensemble learning principles to improve the accuracy and robustness in the performance of the model. The overfitting is mitigated and further enhances generalization through combining multiple base learners trained on bootstrap samples, making a strong tool to defend against false data injection attacks.

4. Results

We utilized the Oak Ridge National Laboratory dataset [15] to illustrate how false data injection attacks (FDIAs) affect power data. We utilized the Python tool to do this experiment and trained different machine learning models. The general overall performance of the model is measured through accuracy. To further analyze the false positives, as they can lead to unnecessary action such as triggering alarms, we measure the precision of each model. Similarly, recall is measured as it is crucial for a false data injection attack; missing an attack (false negative) could have severe consequences for smart grids, such as the undetected manipulation of data or control signals. For a false data injection attack, where the dataset is imbalanced, F1 scores are particularly effective in measuring the balance between precision and recall, ensuring the model does not trade off one for the other. We have used accuracy, precision, recall and the F1 score as the evaluation metrics, which are defined in the equations below:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{FP} + \text{TN} + \text{FN}} \quad (12)$$

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (13)$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (14)$$

$$\text{F1 score} = \frac{2\text{TP}}{2\text{TP} + \text{FP} + \text{FN}} \quad (15)$$

where, TP, TN, FP, and FN are true positive, true negative, false positive, and false negative, respectively.

We have presented the performance comparison of all of the models while testing the model in Table 2. In the table, Extra Tree Classifier (ETC), Random Forest (RF), Extreme Gradient Boosting (XGB), Logistic Regression (LR), Decision Tree (DT), and Bagging Classifier (BC) are denoted as mentioned in parenthesis. We also plot the ROC for a performance comparison of six classifiers, as shown in Figure 4, for an easier comparison of the model's performances. Notably, we see that the Extra Tree, Random Forest, Extreme Gradient Boosting, and Bagging Classifiers have a training set score of 1.00, which indicates a perfect fitting of training data. However, this can also lead to overfitting in some cases, which means the model memorizes the training data well and may not be able to generalize for unseen data. The Decision Tree gives a training score of 0.87, indicating that it performs well on the training dataset, though not as well as previous models. Its depth or structure could be a potential cause for this, as it might be limited to avoid overfitting. Logistic regression has a training score set of 0.77, which is the least in comparison to other models. Due to linearity, it may struggle to comprehend complex data. We used a 70/30 training–test data split to evaluate the model's performance, ensuring that 30% of the data were reserved for testing. To further validate the model and check for overfitting, we applied a 5-fold cross-validation on the training set, dividing the 70% split of training data into five smaller subsets (folds), and training the model on 4 folds and validating it on the 5th fold. This process is repeated for each fold, providing an average validation score across all folds.

The model showed consistent accuracy across the training and test sets, with a minimal performance drop, indicating strong generalization and low overfitting tendencies.

Table 2. Test results of different models' performance.

Metrics	ETC	RF	XGB	LR	DT	BC
Train Set Score	1.00	1.00	1.00	0.77	0.87	1.00
Accuracy Score	0.98	0.97	0.97	0.77	0.95	0.95
Precision Score	0.96	0.96	0.95	0.38	0.92	0.92
Recall Score	0.94	0.90	0.89	0.15	0.85	0.85
F1 score	0.95	0.93	0.92	0.21	0.89	0.89

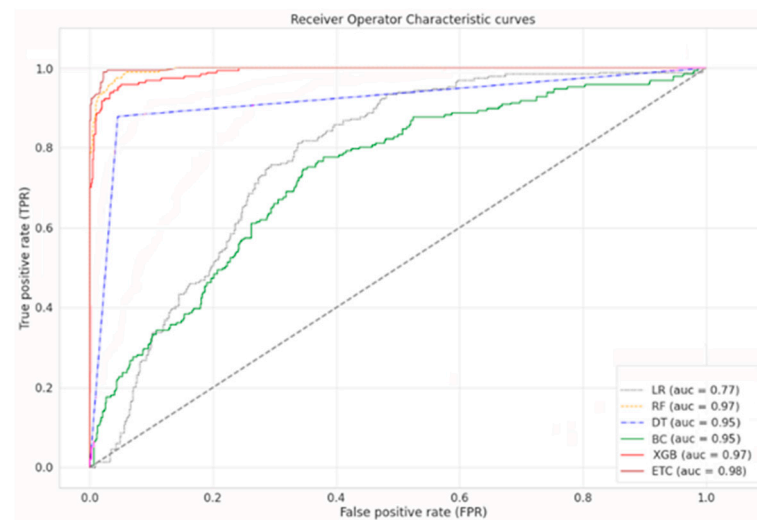


Figure 4. Comparison of ROC curves with different classifiers.

The number of correctly classified instances among all of the classifications is measured by the accuracy of the model, denoted in Equation (12). Through testing the models, we observe that the tree-based models perform exceptionally well in classifying attack and normal events. Extra Tree, Random Forest, and Extreme Gradient Boosting Classifiers give an accuracy of 98%, 97%, and 97%, respectively. The accuracy validates the training set score of these models through aligning with the overfitting tendencies. The Decision Tree and Bagging Classifiers show slightly less accuracy than the previous ones, though they performed well with 95% accuracy, which denotes that these models are highly effective for low-complexity datasets. It is to be noted that, due to the complex nonlinear relationship in the data, linear regression fails to perform well as it struggles with the nonlinear boundaries. To show the correctly classified and incorrectly classified events, we present the confusion matrix for each model performance in Figure 5. It is a useful tool that breaks down the actual versus predicted classification, leveraging the identification of the strengths and weaknesses of the model in distinguishing between classes. In this work, the binary classification scenario detects attack scenarios and normal behavior compromising of four components: true positives (TPs), i.e., attacks classified as attacks correctly; true negatives (TNs), i.e., normal cases classified as normal correctly; false positives (FP), i.e., normal cases misclassified as attacks; and false negatives (FN), i.e., attacks scenarios misclassified as normal.

The ensemble model performs better than the other model in precision as well, which denotes that they possess the capability to classify the positive class and have a low false positive rate. The Extra Tree, Random Forest, and Extreme Gradient Boosting Classifiers are 96%, 96%, and 95% precise. Following these model performances, the Decision Tree and Bagging Classifiers also perform well, indicating a good balance in the tradeoff between true positives and false positives. The formula to calculate the precision is given in Equation (13). In total, 38% of Logistic Regression suggests that a high percentage of the positive predictions it makes are actually false positives. The extremely low precision

points to the struggle of distinguishing between the classes correctly by the model. As seen from the experimental results that the Extra Tree Classifier outperforms other models, the Extra Tree Classifier has an advantage over other models due to its random feature sampling and its ability to aggregate diverse decision rules. Unlike Logistic Regression, which assumes linear relationships between input features and the output, the Extra Tree Classifier does not rely on such assumptions, making it well suited for capturing the complex, non-linear patterns often found in smart-grid data. The Extra Tree Classifier also introduces randomness in feature splits at each node, which reduces the correlation between individual trees within ensembles, in contrast to the Random Forest and Bagging Classifiers, which also employ decision trees. The Extreme Gradient Boosting approach focusing on difficult-to-predict instances sometimes leads to overfitting in complex datasets, thereby underperforming in unseen datasets.

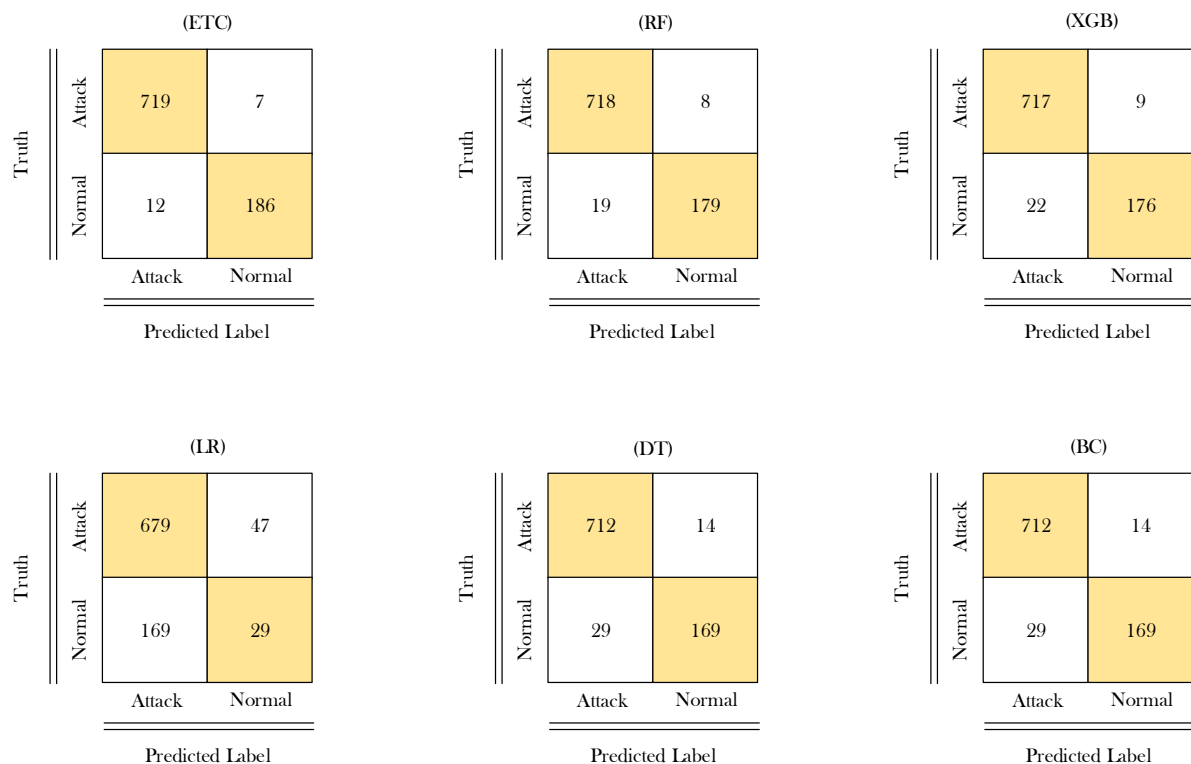


Figure 5. Confusion matrix showing TP, TN, FP, and FN.

We observed that low-impact false data injection attacks are more prone to misclassification due to their overlapping characteristics with normal operational data as the difference is subtle and difficult to distinguish for the model. Features such as voltage phase angle or magnitude, which are minimally altered during low-impact false data injection attacks, lead to confusion between normal operation and the attack scenario. Features like frequency delta, current phase angles, or impedance values hold high impact and, therefore, tend to be differentiated easily and are likely to be misclassified as they have a high impact on power system parameters. The misclassification of high-impact attacks is less likely to happen but can cause an unnecessary shutdown of the system, whereas successful low-impact attacks have less-severe consequences but, overtime, lead to data integrity or unnoticed disruption.

The computational efficiency of the Extra Tree Classifier is relatively faster and can handle larger datasets due to their inherent randomness, but its scalability is limited by the structure of the data and the number of trees. However, Random Forest models can scale well with an increase in data size, but their performance can plateau due to the increased training time and resource demands as the number of trees grows. Extreme Gradient Boosting's ability to parallel processes allows us to efficiently utilize computational

resources, which enables the ability to scale up large data samples efficiently. Extreme Gradient Boosting generally outperforms both Extra Tree and Random Forest in terms of computational efficiency and scalability, particularly for large and complex datasets, especially in real-world scenarios involving significant amounts of data.

Through recall metrics, we look over the proportion of actual positives correctly classified. Models with higher recall indicate fewer false negatives. The Extra Tree, Random Forest, and Extreme Gradient Boosting Classifiers performed well in capturing the truest positives, with very few false negatives. The Decision Tree and Bagging Classifier show less recall than the ensemble model, which means some true positive instances might be missed. Logistic regression downs the recall performance as well; extremely low recall indicates that it misses many true positive instances. Equation (14) presents the mathematical formula to calculate the recall.

To calculate the F1 score, we use Equation (15), which gives the harmonic mean of recall and precision. By balancing both metrics, it accounts for both false positives and false negatives and gives a single metric. The models Extra Tree, Random Forest, and Extreme Gradient Boosting, have high F1 scores, indicating a strong balance between precision and recall. As the pattern follows, Logistic Regression has a low F1 score as well; this indicates a poor overall performance. Due to the low precision and low recall, its ability to classify positive classes is severely compromised. In Figures 6 and 7, a visual of the performance is presented in the form of a line graph and a bar graph, respectively.

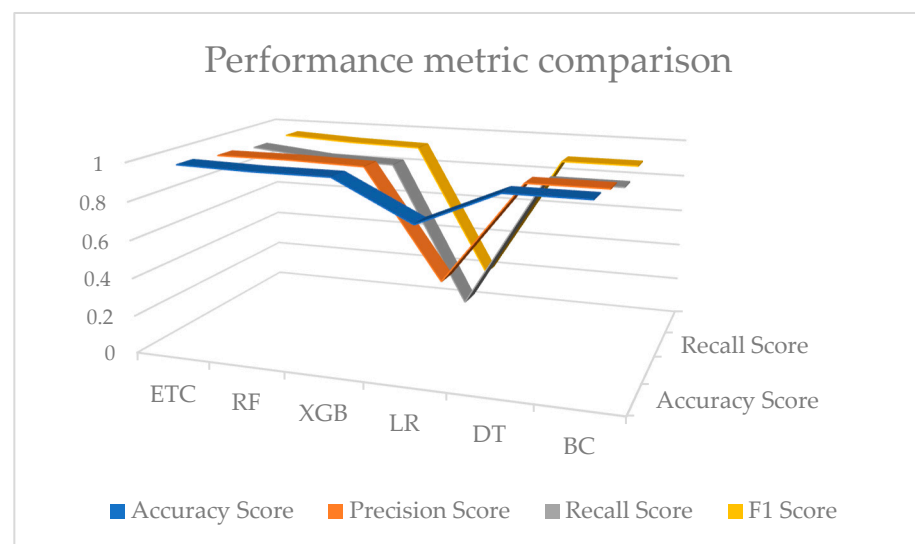


Figure 6. Line graph of performance.

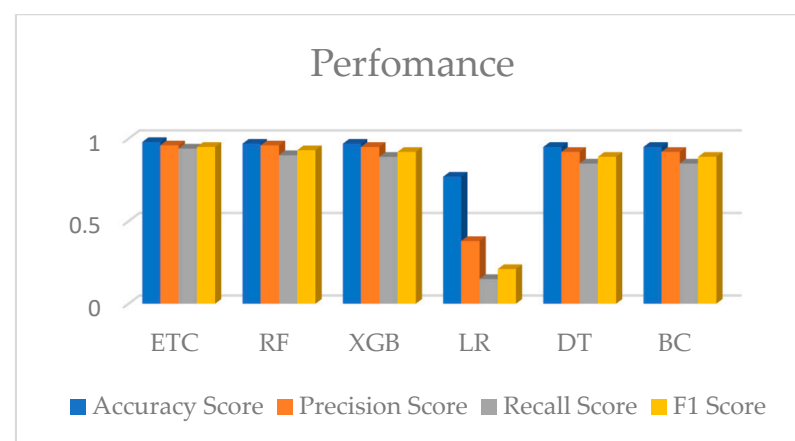


Figure 7. Depicts the performance of different techniques.

5. Discussion

5.1. Data Description

The dataset is collected by the Oak Ridge National Laboratory [15]. The power system shown in Figure 8 is used as an example to classify the data for detection, which is widely used in the literature [43–49]. The dataset utilized in this research consists of three subsets derived from an initial dataset containing fifteen sets, each comprising 37 power system event scenarios. These scenarios are generated within a framework depicting a power system configuration, as illustrated in Table 3.

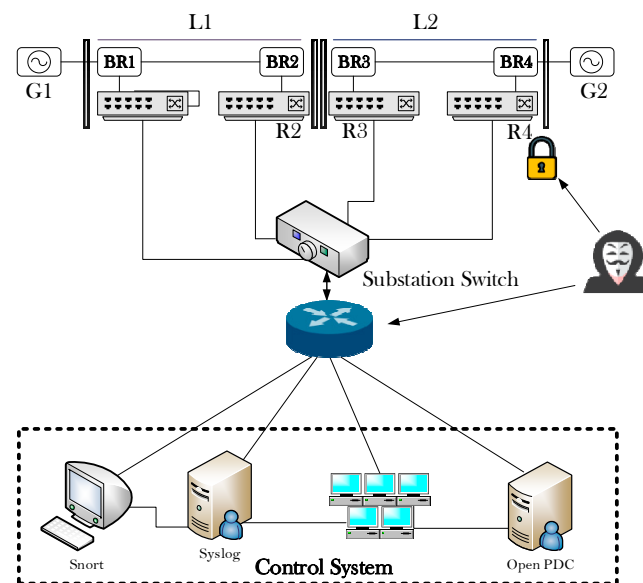


Figure 8. The network topology.

Table 3. Types of attack scenarios and their cause.

Scenario	Description
1–6	Natural event fault at L1 and L2
13–14	Natural event line maintenance
7–12	Data injection—SLG fault replay
15–20	Remote tripping command injection
21–40	Attack event—replay setting change
41	No event—normal operation

The power system framework comprises several components, each playing a crucial role in the generation, distribution, and protection of electrical power [49]. The components, which include generators (G1 and G2), are the primary sources of electrical power within the system. Intelligent Electronic Devices (IEDs) (R1 through R4) are devices responsible for controlling the breakers (BR1 through BR4) within the system. Each Intelligent Electronic Device (IED) is associated with a specific breaker and utilizes a distance protection scheme to trip the breaker upon detecting faults. Breakers (BR1 through BR4) are devices designed to interrupt or break the flow of the electrical current in the event of a fault or overload. Lines (line one and line two) are the transmission lines that carry electrical power between various components of the system. Line one spans from breaker one (BR1) to breaker two (BR2), while line two spans from breaker three (BR3) to breaker four (BR4), as shown in Figure 8.

The dataset is partitioned into three distinct subsets based on the nature of the power system events: 8 scenarios of natural events representing natural occurrences within the power system, such as fluctuations in demand or environmental factors; 1 scenario of no events reflecting a normal operating condition of the power system without any significant events or disturbances; and 28 scenarios of attack events that simulate fault data injection

attacks aimed at disrupting the system operation or causing damage as listed in Table 3. The datasets are randomly sampled at one percent from the initial dataset, ensuring a representative selection of scenarios for analysis and experimentation. Operators have the capability to manually issue commands to the Intelligent Electronic Devices (IEDs) (R1 through R4) to trip the breakers (BR1 through BR4) in case of maintenance or other system requirements. This manual overload feature supplements the automatic protection scheme implemented by the Intelligent Electronic Devices (IEDs). The comprehensive nature of the dataset, coupled with its diverse range of scenarios and event classifications, provides a robust foundation for the evaluation and comparison of machine learning classification techniques for fault data injection attack detection within power systems. Each sub-dataset consists of 128 features, including data gathered from four phasor measurement units (PMUs), Snort alarms, and system logs. Each PMU records 29 variables, while the remaining 12 features from the control panel correspond with the log messages. Table 4 provides a detailed list of feature names and descriptions. Based on our experiments, several features have been identified as crucial for enhancing the detection accuracy of fault data injection attacks playing pivotal roles; for instance, voltage and current phase magnitude are fundamental in identifying anomalies. It significantly fluctuates during attacks, making them vital indicators for detection. Phase angles help in detecting deviations from expected operating conditions during an attack scenario.

Table 4. Different features of the dataset and description.

Features	Description
PA1: VH–PA3: VH	Phase A–C voltage angle
PM1: V–PM3: V	Phase A–C voltage magnitude
PA4: IH–PA6: IH	Phase A–C current angle
PM4: I–PM6: I	Phase A–C current magnitude
PA7: VH–PA9: VH	Positive, negative, and zero-sequence voltage angle
PM7: V–PM9: V	Positive, negative, and zero-sequence voltage magnitude
PA10: VH–PA12: VH	Positive, negative, and zero-sequence current angle
PM10: V–PM12: V	Positive, negative, and zero-sequence current magnitude
F	Relay frequency
DF	Relay frequency delta (rate of change of frequency—dF/dt)
PA: Z	Relay apparent impedance
PA: ZH	Relay apparent impedance angle
S	Relay status indicator

5.2. Data Cleaning

Building high-quality machine learning models has long depended on data cleaning, as machine learning applications are only as good as the quality of the data they are trained on. To make things even more difficult, data are sometimes tainted by missing, inconsistent, or erroneous features brought on by malfunctioning hardware, software, sensors, or time delays. As a result, while adjusting the model’s features and parameters is a big part of the iterative process, finding and cleaning potentially contaminated data is a big component of it. In this study, we have employed different techniques to clean the data for each model independently; through the K-nearest neighbor (KNN) technique, we handle the missing data from our dataset as it is easily implemented and handles any types of missing data [50]. Although the K-nearest neighbor technique requires a large sample size to produce a stable missing data estimate, it was satisfactory for our dataset. Furthermore, normalization techniques such as min–max have a range of 0–1 and standardization with a mean of 0 and a standard deviation of 1 for different algorithms, as shown in Table 5.

Table 5. Data cleaning performed and assumptions. ✔ implies technique we have utilized in data cleaning and ✘ implies we have not utilized in different models.

Model	Handling Missing Values	Normalization	Outlier Removal	Assumptions
Extra Tree Classifier	✔	✘	✘	No

Table 5. Cont.

Model	Handling Missing Values	Normalization	Outlier Removal	Assumptions
Random Forest	✓	Min-max scaling	✗	No
XGBoost	✓	Standardization	Winsorization to cap extreme	No
Logistic Regression	✓	Range 0–1 min-max scaling	IQR (interquartile range)	Linear relation b/w independent and dependent variable
Decision Tree	✓	Standardization mean = 0, std = 1	✗	No
Bagging Classifier	✓	Standardization mean = 0, std = 1	✗	No

We notice through testing our model that ensemble tree classifiers are inherently more tolerant to outliers than Logistic Regression and Extreme Gradient Boosting; therefore, we have removed the outlier from the data using the Winsorization [51] and IQR [52] methods, respectively. For Logistic Regression, the assumption of linearity stems from its mathematical foundation as it models the log-odds of the dependent variable as a linear combination of the independent variables. This assumption is necessary for avoiding misclassification or underfitting.

5.3. Comprehensive Analysis

The dataset used in this work was collected by the Oak Ridge National Laboratory [15]. The power system shown in Figure 3 was used as an example to classify the data for detection, which is widely used in the literature [43–49]. We compare and analyze the other literature in this section. Pan, H. [43] created an inception network model to identify various data types in smart grids that face FDIAs. They used AB-SMOTE to oversample the dataset in order to balance the number of distinct sample types. The inception network and a few common detection models (2D CNN, neural network, additional trees, and CKS-FCS-FLGB) are utilized in this work. Pan, Shengyi. [44] worked on a hybrid IDS, which learns temporal state-based requirements for power system situations; this study proposes a systematic and automated approach. He, Yuchao. [45] presented an improved CNN-GRU model for attack detection through the extraction of the temporal and spatial characteristics from the measured grid data. In Guo, Feng’s [46] study, Extreme Gradient Boosting is proposed to defend the power grid, and the accuracy is combined with SVM. Varmaziari. H [47] used state estimation and machine learning (ML) algorithms to detect the type, location, and attack in power grids. Qu, Z. [48] proposed improved AKF and GRU-CNN neural networks that work as hybrid models for active and passive detection, respectively, for FDIAs. Lastly, Pan. H. [49] also worked on FDIA detection utilizing the PCNN-based image encoding method. In the author’s work, they describe three methods, GAF, MTF, and RP, to encode the relevant features of a power system’s time-series measurement data into 2D images. Table 6 illustrates the result accuracy reported by the authors with different approaches following Figure 9, comprising different states of the art, including our proposed six models. It is seen that the ETC, RF, and XGB have outperformed the other models in terms of accurately classifying the attack and no-attack events.

Table 6. Overview of proposed approaches and their accuracies.

Works	Proposed Approach	Accuracy
[43]	Inception network model for classification	96%
[44]	Hybrid IDS that learns temporal state-based specifications using common data mining technique	90.4%
[45]	GRU—convolution neural network	>93%
[46]	Extreme Gradient Boosting (XGB)	96.33%

Table 6. Cont.

Works	Proposed Approach	Accuracy
[47]	State-estimation based machine learning technique	KNN—95.7% SVM (MLP)—97% SVM (RBF)—90%
[48]	AKF (passive detection) and GRU-CNN (active detection) are mixed in parallel operation	97.5%
[49]	Parallel convolutional neural network (PCNN) detection model based on image data	93.50%

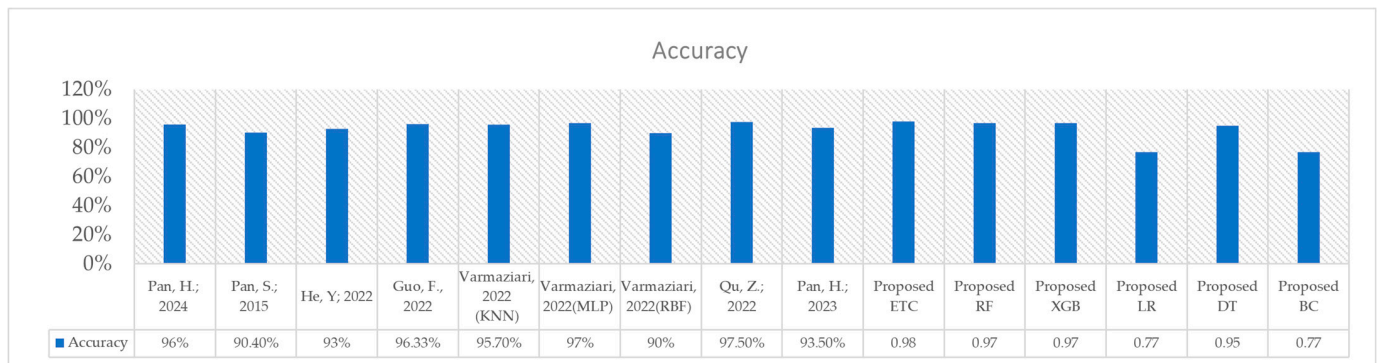


Figure 9. Comparison of accuracy of different states of the art, from left [44–49], and our proposed models.

6. Conclusions

This work addresses the critical issue of detecting and mitigating false data injection attacks in the realm of cyber security to safeguard smart grids through different machine learning approaches. We applied six classifiers to detect the attack events and normal events, including the Extra Tree, Random Forest, Extreme Gradient Boosting, Logistic Regression, Decision Tree, and Bagging Classifiers, for the sole purpose of building a robust detection system that is highly capable of detecting dust between false data injection attack (FDIA) events and normal events in a smart grid environment. We have rigorously evaluated the performance of all of the classifiers in terms of their accuracy, precision, recall, and F1 score, providing a well-rounded assessment of their performance and capabilities.

The results prove that ensemble models such as the Extra Tree Classifier, Random Forest Classifier, and Extreme Gradient Boosting Classifier stand out among other techniques to detect the complex pattern of false data injection while maintaining the false positive rates, showcasing their high effectiveness in false data injection attack (FDIA) scenarios through handling the high-dimensional and dynamic nature of smart grid data. We have compared our techniques with other conducted works on the same dataset, further validating the strength of our work, as the ensemble models, such as the Extra Tree, Random Forest, and Extreme Gradient Boosting Classifiers, gave more accurate results, providing a promising solution to the false data injection attack (FDIA) problem. By comparison, the potential of integrating the ensemble learning methods to extend the boundaries of cybersecurity with the smart grid ecosystem is highlighted. Furthermore, the research could focus on refining these models through a wider range of attack vectors and operational conditions. In addition, the real-time implementation of these models into a large smart grid infrastructure will move towards a more resilient and secure energy system.

Future direction: This research can be extended using several other advanced techniques for detecting false data injection attacks (FDIAs), leveraging the complexity of the scenarios. Integrating deep learning architectures, such as recurrent neural networks (RNNs) and transformers, could enhance detection accuracy by capturing complex temporal dependencies and patterns over time in streaming data from smart grid environments. Federated learning and privacy-preserving machine learning methods could be implemented to allow distributed detection across multiple nodes while maintaining data

privacy and security, which is critical in real-world smart grid applications where data sensitivity is paramount. Furthermore, reinforcement techniques that can be used to develop an autonomous agent that can detect and respond to false data injection attacks (FDIAs) in real time could provide a proactive layer of security by optimizing countermeasures and adapting to various attack scenarios. Finally, evaluating real-time, large-scale smart grid simulations with diverse operational conditions and deploying the models in live smart-grid testbeds would provide critical insights into their operational feasibility, latency, and scalability, ultimately moving towards a robust and resilient cybersecurity infrastructure for smart grids.

Author Contributions: Conceptualization, A.S., M.T. and A.I.S.; Formal Analysis, A.S., M.T. and A.I.S.; Funding Acquisition, A.I.S.; Investigation, A.S. and M.T.; Methodology, A.S. and M.T.; Project Administration, A.I.S.; Resources, A.I.S.; Supervision, A.I.S.; Validation, A.S. and M.T.; Writing—Original Draft, A.S.; Writing—Review and Editing, M.T. and A.I.S. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported in part by the U.S. Department of Energy (DOE) under grant number DE-NA0004109.

Data Availability Statement: The dataset used in this study is openly available and was prepared and generated by the Oak Ridge National Laboratory (ORNL) in Tennessee, United States. It can be accessed through the reference cited within the paper for further details and usage.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Esmalifalak, M.; Liu, L.; Nguyen, N.; Zheng, R.; Han, Z. Detecting stealthy false data injection using machine learning in smart grid. *IEEE Syst. J.* **2014**, *11*, 1644–1652. [CrossRef]
2. Shees, A.; Hussain, M.T.; Tariq, M.; Sarwar, A.; Sarwat, A.I. Integration of Artificial Intelligence in Future Smart Grids: An LSTM-RNN Based Approach for Optimizing Energy Efficiency in Smart Grids. In Proceedings of the 2023 IEEE International Conference on Energy Technologies for Future Grids (ETFG), Wollongong, NSW, Australia, 3–6 December 2023; IEEE: Piscataway, NJ, USA, 2023.
3. Ali, M.N.; Amer, M.; Elsis, M. Reliable IoT paradigm with ensemble machine learning for faults diagnosis of power transformers considering adversarial attacks. *IEEE Trans. Instrum. Meas.* **2023**, *72*, 3525413. [CrossRef]
4. Elsis, M.; Su, C.-L.; Ali, M.N. Design of reliable IoT systems with deep learning to support resilient demand side management in smart grids against adversarial attacks. *IEEE Trans. Ind. Appl.* **2023**, *60*, 2095–2106. [CrossRef]
5. Lu, K.-D.; Wu, Z.-G. Multi-objective false data injection attacks of cyber–physical power systems. *IEEE Trans. Circuits Syst. II Express Briefs* **2022**, *69*, 3924–3928. [CrossRef]
6. Taher, M.A.; Behnamfar, M.; Sarwat, A.; Tariq, M. False Data Injection Attack Detection and Mitigation Using Non-Linear Autoregressive Exogenous Input-Based Observers in Distributed Control for DC Microgrid. *IEEE Open J. Ind. Electron. Soc.* **2024**, *5*, 441–457. [CrossRef]
7. Case, D.U. Analysis of the cyber-attack on the Ukrainian power grid. *Electr. Inf. Shar. Anal. Cent. (E-ISAC)* **2016**, *388*, 3.
8. Available online: <https://www.cyberdefensemagazine.com/cybersecurity-in-critical-infrastructure-protecting-power-grids-and-smart-grids/> (accessed on 28 October 2024).
9. Mo, Y.; Chabukwar, R.; Sinopoli, B. Detecting integrity attacks on SCADA systems. *IEEE Trans. Control. Syst. Technol.* **2013**, *22*, 1396–1407. [CrossRef]
10. Liu, Y.; Ning, P.; Reiter, M.K. False data injection attacks against state estimation in electric power grids. *ACM Trans. Inf. Syst. Secur. (TISSEC)* **2011**, *14*, 1–33. [CrossRef]
11. Deng, R.; Xiao, G.; Lu, R.; Liang, H.; Vasilakos, A.V. False data injection on state estimation in power systems—Attacks, impacts, and defense: A survey. *IEEE Trans. Ind. Inform.* **2016**, *13*, 411–423. [CrossRef]
12. Cui, S.; Han, Z.; Kar, S.; Kim, T.T.; Poor, H.V.; Tajer, A. Coordinated data-injection attack and detection in the smart grid: A detailed look at enriching detection solutions. *IEEE Signal Process. Mag.* **2012**, *29*, 106–115.
13. Lu, X.; Jing, J.; Wu, Y. False Data Injection Attack Location Detection Based on Classification Method in Smart Grid. In Proceedings of the 2020 2nd International Conference on Artificial Intelligence and Advanced Manufacture (AIAM), Manchester, UK, 15–17 October 2020; pp. 133–136.
14. Taher, A.M.; Tariq, M.; Sarwat, A.I. Trust-Based Detection and Mitigation of Cyber Attacks in Distributed Cooperative Control of Islanded AC Microgrids. *Electronics* **2024**, *13*, 3692. [CrossRef]
15. Morris, T. Industrial Control System (ICS) Cyber-Attack Datasets. Available online: <https://sites.google.com/a/uah.edu/tommy-morris-uah/ics-data-sets> (accessed on 21 September 2024).
16. Yohanandhan, R.V.; Elavarasan, R.M.; Manoharan, P.; Mihet-Popa, L. Cyber-Physical Power System (CPPS): A Review on Modeling, Simulation, and Analysis with Cyber Security Applications. *IEEE Access* **2020**, *8*, 151019–151064. [CrossRef]

17. Nejabatkhah, F.; Li, Y.W.; Liang, H.; Ahrabi, R.R. Cyber-security of smart microgrids: A survey. *Energies* **2021**, *14*, 27. [\[CrossRef\]](#)
18. Ye, J.; Giani, A.; Elasser, A.; Mazumder, S.K.; Farnell, C.; Mantooth, H.A.; Kim, T.; Liu, J.; Chen, B.; Seo, G.-S.; et al. A Review of Cyber-Physical Security for Photovoltaic Systems. *IEEE J. Emerg. Sel. Top. Power Electron.* **2021**, *10*, 4879–4901. [\[CrossRef\]](#)
19. Hossain, E.; Khan, I.; Un-Noor, F.; Sikander, S.S.; Sunny, S.H. Application of Big Data and Machine Learning in Smart Grid, and Associated Security Concerns: A Review. *IEEE Access* **2019**, *7*, 13960–13988. [\[CrossRef\]](#)
20. Alimi, O.A.; Ouahada, K.; Abu-Mahfouz, A.M. A Review of Machine Learning Approaches to Power System Security and Stability. *IEEE Access* **2020**, *8*, 113512–113531. [\[CrossRef\]](#)
21. Musleh, A.S.; Chen, G.; Dong, Z.Y. A Survey on the Detection Algorithms for False Data Injection Attacks in Smart Grids. *IEEE Trans. Smart Grid* **2019**, *11*, 2218–2234. [\[CrossRef\]](#)
22. Kotsiopoulos, T.; Sarigiannidis, P.; Ioannidis, D.; Tzovaras, D. Machine Learning and Deep Learning in smart manufacturing: The Smart Grid paradigm. *Comput. Sci. Rev.* **2021**, *40*, 100341. [\[CrossRef\]](#)
23. Cui, L.; Qu, Y.; Gao, L.; Xie, G.; Yu, S. Detecting false data attacks using machine learning techniques in smart grid: A survey. *J. Netw. Comput. Appl.* **2020**, *170*, 102808. [\[CrossRef\]](#)
24. Jow, J.; Xiao, Y.; Han, W. A survey of intrusion detection systems in smart grid. *Int. J. Sens. Networks* **2017**, *23*, 170–186. [\[CrossRef\]](#)
25. Radoglou-Grammatikis, P.I.; Sarigiannidis, P.G. Securing the Smart Grid: A Comprehensive Compilation of Intrusion Detection and Prevention Systems. *IEEE Access* **2019**, *7*, 46595–46620. [\[CrossRef\]](#)
26. Mahesh, B. Machine learning algorithms—A review. *Int. J. Sci. Res. (IJSR)* **2020**, *9*, 381–386. [\[CrossRef\]](#)
27. Tufail, S.; Riggs, H.; Riggs, H.; Sarwat, A.I. Advancements and challenges in machine learning: A comprehensive review of models, libraries, applications, and algorithms. *Electronics* **2023**, *12*, 1789. [\[CrossRef\]](#)
28. Alsariera, Y.A.; Adeyemo, V.E.; Balogun, A.O.; Alazzawi, A.K. AI meta-learners and extra-trees algorithm for the detection of phishing websites. *IEEE Access* **2020**, *8*, 142532–142542. [\[CrossRef\]](#)
29. Chen, C.; Wang, N.; Chen, M. Prediction model of end-point phosphorus content in consteel electric furnace based on PCA-extra tree model. *ISIJ Int.* **2021**, *61*, 1908–1914. [\[CrossRef\]](#)
30. Chakrabarty, N.; Biswas, S. Navo minority over-sampling technique (NMOTe): A consistent performance booster on imbalanced datasets. *J. Electron. Inform.* **2020**, *2*, 96–136. [\[CrossRef\]](#)
31. Sina, Y.; Yin, S.; Gibran, A.M. Intelligent fault diagnosis of manufacturing processes using extra tree classification algorithm and feature selection strategies. *IEEE Open J. Ind. Electron. Soc.* **2023**, *4*, 618–628.
32. Wu, B.; Zhang, B.; Li, W.; Jiang, F. A novel method for remaining useful life prediction of bearing based on spectrum image similarity measures. *Mathematics* **2022**, *10*, 2209. [\[CrossRef\]](#)
33. Zhao, Y.; Wen, J.; Xiao, F.; Yang, X.; Wang, S. Diagnostic Bayesian networks for diagnosing air handling units' faults—Part I: Faults in dampers, fans, filters and sensors. *Appl. Therm. Eng.* **2017**, *111*, 1272–1286. [\[CrossRef\]](#)
34. Pal, M. Random Forest classifier for remote sensing classification. *Int. J. Remote Sens.* **2005**, *26*, 217–222. [\[CrossRef\]](#)
35. Breiman, L. Bagging predictors. *Mach. Learn.* **1996**, *24*, 123–140. [\[CrossRef\]](#)
36. Breiman, L. Random forests. *Mach. Learn.* **2001**, *45*, 5–32. [\[CrossRef\]](#)
37. Briem, G.; Benediktsson, J.; Sveinsson, J. Multiple classifiers applied to multisource remote sensing data. *IEEE Trans. Geosci. Remote. Sens.* **2002**, *40*, 2291–2299. [\[CrossRef\]](#)
38. Chan, J.C.-W.; Paelinckx, D. Evaluation of Random Forest and Adaboost tree-based ensemble classification and spectral band selection for ecotope mapping using airborne hyperspectral imagery. *Remote. Sens. Environ.* **2008**, *112*, 2999–3011. [\[CrossRef\]](#)
39. Pal, M.; Mather, P.M. An assessment of the effectiveness of decision tree methods for land cover classification. *Remote. Sens. Environ.* **2003**, *86*, 554–565. [\[CrossRef\]](#)
40. Hastie, T.; Tibshirani, R.; Friedman, J. *Random Forests, The Elements of Statistical Learning*; Springer: New York, NY, USA, 2009; pp. 587–604.
41. Pedregosa, F.; Varoquaux, G.; Gramfort, A.; Michel, V.; Thirion, B.; Grisel, O.; Blondel, M.; Prettenhofer, P.; Weiss, R.; Dubourg, V.; et al. Scikit-learn: Machine learning in Python. *J. Mach. Learn. Res.* **2011**, *12*, 2825–2830.
42. Wu, Z.; Zhou, M.; Lin, Z.; Chen, X.; Huang, Y. Improved genetic algorithm and XGBoost classifier for power transformer fault diagnosis. *Front. Energy Res.* **2021**, *9*, 745744. [\[CrossRef\]](#)
43. Pan, H.; Yang, H.; Na, C.N.; Jin, J.Y. Multi-data classification detection in smart grid under false data injection attack based on Inception network. *IET Renew. Power Gener.* **2024**, *18*, 2430–2439. [\[CrossRef\]](#)
44. Pan, S.; Morris, T.; Adhikari, U. Developing a hybrid intrusion detection system using data mining for power systems. *IEEE Trans. Smart Grid* **2015**, *6*, 3104–3113. [\[CrossRef\]](#)
45. He, Y.; Li, L.; Qian, H.; Qian, H. CNN-GRU based fake data injection attack detection method for power grid. In Proceedings of the 2022 2nd International Conference on Electrical Engineering and Control Science (IC2ECS), Nanjing, China, 18 December 2022.
46. Guo, F.; Yao, S.; Zhang, N.; He, Y. XGBoost based fake data injection attack detection method for power grid. In Proceedings of the 2022 2nd International Conference on Electrical Engineering and Control Science (IC2ECS), Nanjing, China, 18 December 2022.
47. Varmaziari, H.; Maryam, D. Cyber Attack Detection in PMU Networks Exploiting the Combination of Machine Learning and State Estimation-Based Methods. In Proceedings of the 2021 11th Smart Grid Conference (SGC), Tabriz, Iran, 7–9 December 2021.
48. Qu, Z.; Bo, X.; Yu, T.; Liu, Y.; Dong, Y.; Kan, Z.; Wang, L.; Li, Y. Active and passive hybrid detection method for power CPS false data injection attacks with improved AKF and GRU-CNN. *IET Renew. Power Gener.* **2022**, *16*, 1490–1508. [\[CrossRef\]](#)
49. Pan, H.; Feng, X.; Na, C.; Yang, H. A model for detecting false data injection attacks in smart grids based on the method utilized for image coding. *IEEE Syst. J.* **2023**, *17*, 6181–6191. [\[CrossRef\]](#)

50. Joel, O.L.; Doorsamy, W.; Paul, B.S. A review of missing data handling techniques for machine learning. *Int. J. Innov. Technol. Interdiscip. Sci.* **2022**, *5*, 971–1005.
51. Bruce, B.F. *The SAGE Encyclopedia of Educational Research, Measurement, and Evaluation*; SAGE: Newcastle upon Tyne, UK, 2018.
52. Yang, J.; Rahardja, S.; Fränti, P. Outlier detection: How to threshold outlier scores? In Proceedings of the International Conference on Artificial Intelligence, Information Processing and Cloud Computing, Sanya, China, 19–21 December 2019.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.